BIG DATA ANALYTICS PROJECT

RETAIL DEMAND FORECASTING AND INVENTORY OPTIMIZATION

An integrated analytics workflow using PostgreSQL, DBeaver, Python, Databricks, Delta Lake and Power BI

PROJECT DETAIL	INFORMATION
Student Name	[Enter your name]
Programme / Section	[Enter programme and section]
Institution	[Enter institution name]
Faculty Guide	Dr. Butchi Babu Muvva
Course	Big Data Analytics
Submission Date	[Enter submission date]

Dataset: synthetic retail sales and inventory data | Period: 1 January–29 February 2024

Prepared for academic evaluation. Replace bracketed details and insert screenshots from your own tool runs before final submission.

Declaration, Dataset Provenance & Integrity

This report documents an academic prototype for retail demand forecasting and inventory analysis. The workbook explicitly identifies its records as synthetic/simulated. The figures therefore demonstrate analytical methods and must not be represented as the actual performance of a real retailer.

The report separates three evidence types: (1) measured descriptive statistics computed from the supplied workbook; (2) model metrics stored in the project's Python output; and (3) implementation designs or steps that still require the student to execute in their own environment. A tool is not described as successfully integrated merely because a script or architecture diagram exists.

Declaration

I declare that I have reviewed the dataset, methods, calculations and project artifacts submitted with this report. I will verify all screenshots, execution results, citations and institutional formatting before final submission.

Data and technology integrity

- No customer-level personal data is present in the supplied workbook.
- The dataset is synthetic; external business generalization is not justified.
- Credentials, tokens and workspace secrets must not be committed to GitHub.
- All final claims should be reproducible from the included scripts and data extracts.

Executive Summary

The workbook contains 6,000 transaction-like store–product–date records covering 60 dates, 5 stores, 20 products and 5 cities. The period runs from 01 January 2024 to 29 February 2024.

Indicator	Observed value	Interpretation
Recorded revenue	₹13,969,230.40	Sum of the workbook's Revenue INR field
Units sold	141,189	Sum of Units Sold
Estimated unmet demand	2,015	Sum of supplied Estimated Unmet Demand field
Estimated gross profit	₹2,981,630.40	Revenue minus units sold × unit cost; estimate only
Missing cells	0	Null count across the sales sheet
Exact duplicate rows	0	Full-row duplicate check

The largest city by recorded revenue is Secunderabad (₹3,151,614.20); the largest category is Grocery (₹5,624,615.80). The largest-revenue product is Toor Dal 1kg (₹1,671,008.00).

The proposed architecture uses Python for profiling and forecasting, PostgreSQL for structured storage and SQL analysis, DBeaver as the SQL client, Databricks/Spark for distributed transformation, Delta Lake for reliable lakehouse tables and Power BI for decision-facing dashboards. The project is a prototype: the short two-month synthetic time series and missing operational fields limit the strength of any replenishment recommendation.

Business Problem & Decision Context

Retail demand planning is a balancing problem. Ordering too little can reduce availability and create unmet demand; ordering too much ties up working capital and can increase storage, markdown or spoilage costs. Store-level decisions are more useful when product, date, promotion and stock signals are analyzed together.

Core business question

How can historical store–product sales and available calendar/promotion fields support a repeatable demand forecast and better inventory review?

Decisions the analysis is intended to support

- Which cities, categories and products account for the largest share of recorded revenue?
- How does demand vary across dates and weekdays?
- Do rows marked as promotion show different average unit sales from non-promotion rows?
- Where does the supplied data flag estimated unmet demand or low closing stock?
- Can a simple model outperform a naive baseline on a chronological holdout?

Decision boundary

The report does not claim to automate purchasing. Lead time, supplier reliability, order minimums, shelf life, service-level targets and carrying costs are not supplied. Forecast outputs should trigger review, not purchase orders, until those fields and a controlled pilot are available.

Objectives, Scope & Success Criteria

Objective	Deliverable	Evidence of completion
Profile data quality	Profiling summary	Row counts, types, nulls, duplicates and key checks
Model structured data	PostgreSQL schema	Tables, keys, indexes and row-count reconciliation
Answer business questions	SQL analytics	Saved query outputs that reconcile to Python
Build a baseline forecast	Python pipeline	Chronological split, metrics and prediction file
Demonstrate big-data workflow	Spark/Delta notebook	Successful run, table paths and row counts—or clearly marked plan
Support management decisions	Power BI report	KPI cards, 4+ visuals, slicers and reconciled totals
Make work reproducible	GitHub repository	README, code, dependencies, report and no secrets

In scope

Descriptive analysis, data modelling, SQL, Python baseline modelling, a Spark/Delta design, Power BI dashboard implementation and version-controlled documentation.

Out of scope

Live point-of-sale integration, streaming ingestion, production MLOps, causal proof of promotion effectiveness, guaranteed annual forecasts and automated replenishment.

Dataset Inventory & Structure

Workbook sheet	Rows × columns	Role in analysis
Project Guide	11 × 2	Dataset description, type and project question
Retail Sales Data	6,000 × 18	Primary daily store-product fact-like records
Product Master	20 × 5	Product/category/regular price/unit cost lookup
Store Master	5 × 3	Store/city/store type lookup

Sales sheet fields

Field group	Fields	Use
Keys / time	Date, Store ID, Product ID	Grain, joins, chronological split
Descriptors	City, Product Name, Category, Day of Week	Grouping and slicers
Commercial	Promotion, Unit Price INR, Revenue INR, Unit Cost INR	Promotion comparison and estimated margin
Demand / stock	Units Sold, Opening Stock, Restocked Units, Closing Stock, Estimated Unmet Demand	Demand and inventory review

The intended grain is one observation per date × store × product. This must be tested, not assumed, because duplicate or multiple-promotion records at the same grain would change the correct primary key and aggregation logic.

Data Quality & Validation

Check	Result from supplied sales sheet	Implication
Row count	6,000	Baseline reconciliation target
Date coverage	2024-01-01 to 2024-02-29	Only 60 days of history
Missing cells	0	No nulls found in this sheet
Exact duplicate rows	0	No exact duplicate rows found
Unique stores	5	Compare with Store Master
Unique products	20	Compare with Product Master
Negative units	0	Investigate returns/invalid records
Negative revenue	0	Investigate refunds/invalid records
Revenue arithmetic mismatches	0	Investigate rounding or inconsistent fields

Important caveat: inventory continuity

The workbook provides opening stock, restocked units, closing stock, units sold and estimated unmet demand. These fields should be checked against an inventory equation such as opening stock + receipts − sales − closing stock. Any mismatch should be reported as a data limitation; do not silently repair it without a documented business rule.

Validation before submission

- Confirm one row per date/store/product or document exceptions.
- Check Product ID and Store ID values exist in the master sheets.
- Reconcile totals across Excel, Python, PostgreSQL and Power BI.
- Document every transformation and preserve the original source file unchanged.

Methodology & Analytical Workflow

1. Inspect the workbook and establish data provenance; preserve the source unchanged.
2. Profile columns, data types, nulls, duplicates, key coverage and arithmetic consistency in Python.
3. Create dimension and fact tables in PostgreSQL; use DBeaver to connect, import, query and validate.
4. Create business-focused SQL summaries and compare their totals with Python aggregates.
5. Aggregate demand at a defined grain and evaluate a baseline model using a time-ordered train/test split.
6. Transform raw data in Spark and write Bronze/Silver/Gold Delta tables if the Databricks workspace and permissions are available.
7. Build Power BI pages with consistent measures, filters and clear metric definitions.
8. Package code, instructions, report, screenshots and known limitations in GitHub.

Why this sequence?

Profiling before modelling prevents incorrect results from being hidden by downstream tools. SQL reconciliation establishes a trusted analytical layer. Time-ordered evaluation reduces the risk of using future observations to predict the past. Dashboard totals are only credible when they reconcile to the same source definitions.

Solution Architecture

Layer	Technology	Responsibility	Proof to capture
Source	Excel / CSV	Original data and repeatable extracts	Workbook sheet names and row counts
Validation	Python / pandas	Schema checks, cleaning, descriptive analysis	Run log and output files
Relational store	PostgreSQL	Tables, constraints, indexes and SQL analytics	Schema and query result screenshots
SQL client	DBeaver	Connection, import, query authoring and inspection	Connection and result grid (hide credentials)
Lakehouse compute	Databricks / PySpark	Distributed transformations	Notebook run and row-count output
Storage reliability	Delta Lake	Versioned ACID table format / medallion layers	Delta table listing and successful write
BI layer	Power BI	Interactive dashboards and management measures	Report pages and refreshed totals
Version control	GitHub	Code, docs, dependencies and collaboration	Repository URL and rendered README

Logical flow

Excel/CSV → Python validation → PostgreSQL → DBeaver SQL analysis → Databricks/PySpark → Delta Lake Bronze/Silver/Gold → Power BI → GitHub documentation.

This is the target logical architecture. The final report must distinguish components actually run from those documented as implementation plans. A diagram is not evidence of a successful live connection.

Relational Model & Table Design

A normalized relational design separates descriptive attributes from repeated observations. The suggested model uses dim_product and dim_store dimensions with a fact_retail_sales table at daily store-product grain.

Table	Primary key	Selected columns	Purpose
dim_product	product_id	product_name, category, regular_price_inr, unit_cost_inr	One row per product
dim_store	store_id	city, store_type	One row per store
fact_retail_sales	date + store_id + product_id*	units_sold, revenue_inr, stock fields, promotion	Daily measures and foreign keys

*Use the composite key only after validating grain. If the source contains more than one row per date/store/product, either aggregate intentionally or include a justified transaction/segment key.

Design rationale

- Dimension tables reduce repeated labels and support consistent slicers.
- Foreign keys protect the integrity of product/store references.
- A date column enables chronological reporting and time-based calculations.
- Indexes on date, store_id and product_id can support common filters and joins.

Load order

Create dimensions first, then load fact records. Where source names contain spaces, import into a staging table and map source columns into standardized snake_case names. Compare source and target row counts after every load.

PostgreSQL & DBeaver Implementation

9. Install PostgreSQL and create a dedicated project database.
10. Open DBeaver and create a PostgreSQL connection using the local host, port, database, username and password.
11. Run the schema script to create dimensions, fact table, constraints and indexes.
12. Import Product Master and Store Master first; validate key uniqueness.
13. Import Retail Sales Data into a staging table, map fields and load the fact table.
14. Run row-count, date-range, key-integrity and total-reconciliation queries.
15. Save SQL queries and screenshots in the project repository; never expose passwords in screenshots.

Example analytical questions

- What is revenue and units sold by city and category?
- Which products contribute most revenue within each city?
- What are daily demand trends and weekday differences?
- Which records combine low closing stock with non-zero estimated unmet demand?
- How do average units sold compare between promotion and non-promotion rows?

DBeaver is a database client, not a database engine. PostgreSQL stores and executes the SQL; DBeaver provides the interface to connect, browse and run queries.

SQL KPI Definitions & Reconciliation

KPI	Definition	Caution
Recorded revenue	SUM(revenue_inr)	Use source field; don't recalculate without checking rounding
Units sold	SUM(units_sold)	Quantity, not number of transactions
Estimated gross profit	SUM(revenue_inr − units_sold × unit_cost_inr)	Estimate; excludes operating expenses and other costs
Unmet demand	SUM(estimated_unmet_demand)	Supplied estimate; definition not independently verified
Average selling price	SUM(revenue) / SUM(units sold)	Weighted ratio, not average of row-level prices
Promotion share	Promotion rows / all rows	Row share is not necessarily share of sales or causal lift
Low-stock records	Count of rows meeting an explicit stock threshold	Threshold must be stated and justified

Reconciliation protocol

Compute each KPI in SQL and Python using the same filters and definitions. A difference is a defect to investigate, not a value to average. Check duplicate grain, null handling, integer/decimal conversion, source rounding and join multiplication.

Illustrative query pattern

```
SELECT city, SUM(revenue_inr) AS revenue_inr,
    SUM(units_sold) AS units_sold
FROM fact_retail_sales
GROUP BY city
ORDER BY revenue_inr DESC;
```

Python Data Preparation & Exploratory Analysis

The Python pipeline uses pandas for reading, validation and aggregation, NumPy for feature construction, and scikit-learn for a baseline Random Forest regressor. Outputs include daily aggregates, test predictions and a model-metrics CSV.

Data preparation decisions

- Parse Date as a datetime field and validate the required columns.
- Check exact duplicates and critical missing values before modelling.
- Create calendar features: day of month, weekday, month and weekend flag.
- Encode store, product and category identifiers as model features.
- Aggregate at a documented store-product-date grain before splitting by time.
- Use a fixed random seed for reproducibility and preserve prediction outputs.

Exploratory questions

The analysis examines daily revenue and unit trends, city/category contribution, product rankings, weekday patterns, promotion-labelled records and the relationship between closing stock and the supplied unmet-demand estimate.

Descriptive association is not causal evidence. In particular, promotion rows may be assigned to particular products or dates; a simple comparison cannot isolate the effect of a promotion from those differences.

Revenue Trend Over Time

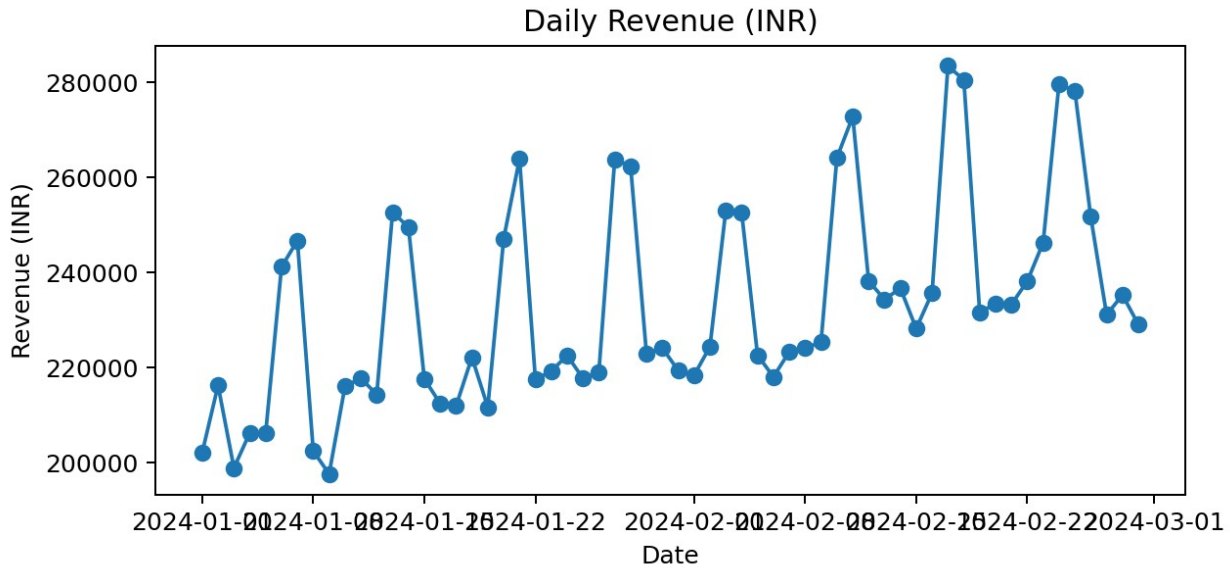


Figure 1. Daily revenue summed across all stores and products in the supplied synthetic workbook.

Total recorded revenue across the observation window is ₹13,969,230.40. The daily series should be inspected for spikes, dips and potential data-generation patterns before drawing operational conclusions.

A short two-month period cannot establish annual seasonality. The plot is useful for describing observed variation and checking data quality, but not for asserting that the same pattern will repeat in future months.

Interpretation checklist

- Check whether unusual peaks align with promotion flags or product mix changes.
- Confirm no dates are missing from the expected 60-day calendar.
- Compare daily totals with SQL output to validate the pipeline.
- Avoid reading a single-day spike as evidence of a persistent trend.

City-Level Revenue Contribution

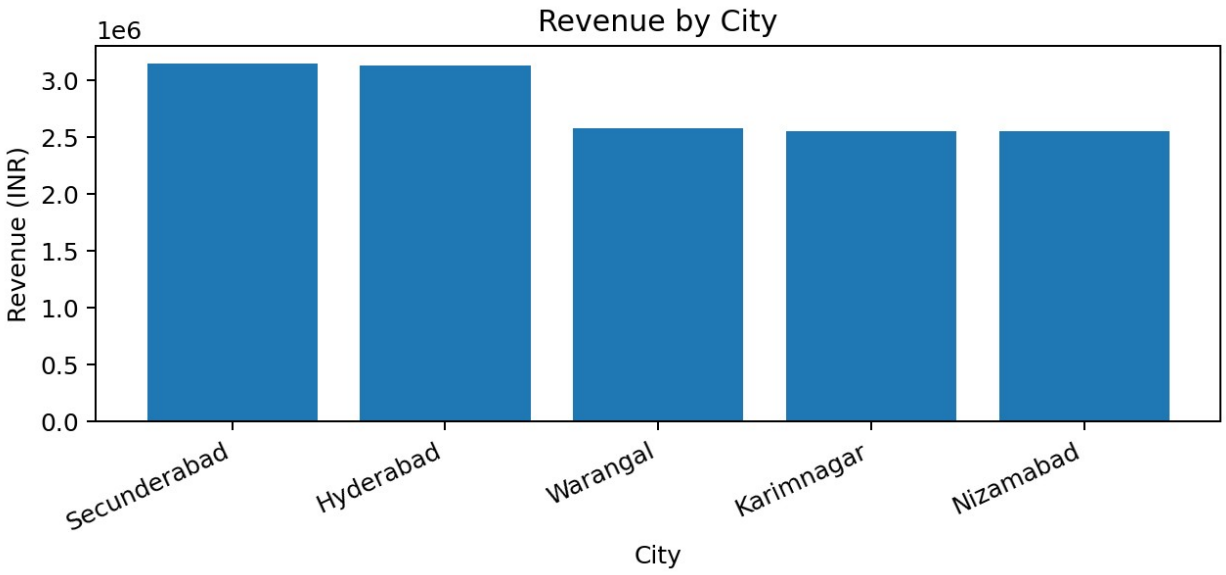


Figure 2. Recorded revenue by city.

Rank	City	Revenue (INR)	Units sold	Unmet demand
1	Secunderabad	₹3,151,614.20	31,572	452
2	Hyderabad	₹3,132,514.50	31,337	392
3	Warangal	₹2,578,499.10	26,173	389
4	Karimnagar	₹2,554,036.50	26,101	376
5	Nizamabad	₹2,552,566.10	26,006	406

Secunderabad is the largest city by recorded revenue in this dataset. This ranking is descriptive and may reflect the synthetic data design, price mix and store/product mix; it should not be treated as evidence of real market share.

Category Mix & Product Contribution

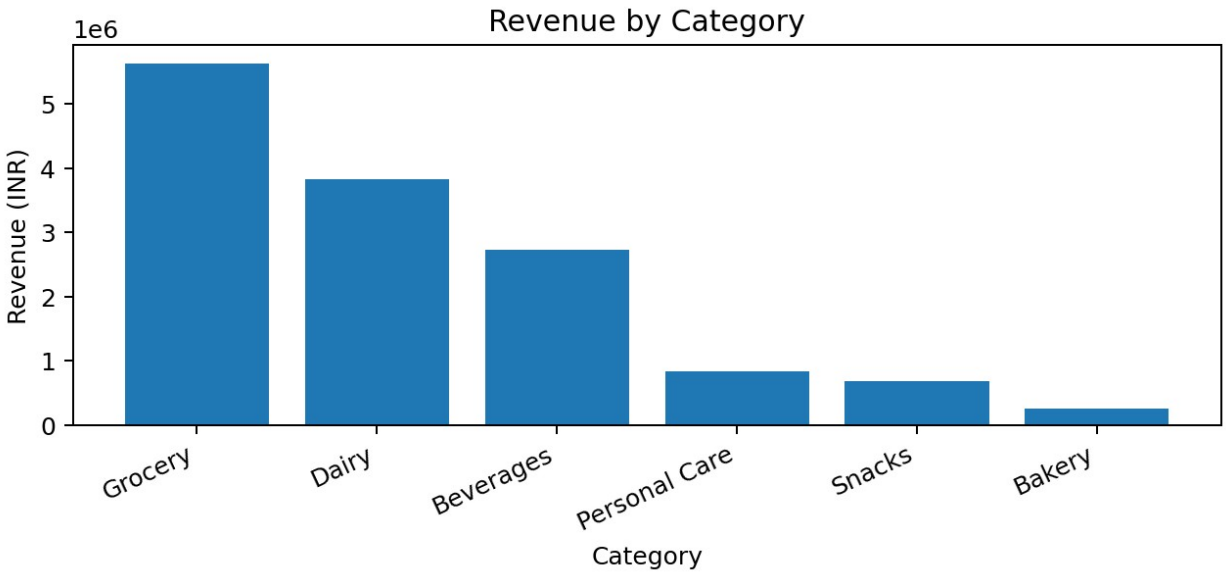


Figure 3. Recorded revenue by category.

Category	Revenue (INR)	Units sold	Unmet demand
Grocery	₹5,624,615.80	33,526	446
Dairy	₹3,817,827.10	47,577	710
Beverages	₹2,733,773.00	24,181	343
Personal Care	₹839,174.10	11,192	182
Snacks	₹691,986.00	19,174	278
Bakery	₹261,854.40	5,539	56

The leading category by revenue is Grocery. Category-level analysis helps prioritize deeper review, but revenue leadership does not automatically mean the highest margin, strongest growth or most urgent inventory need.

Product Prioritization

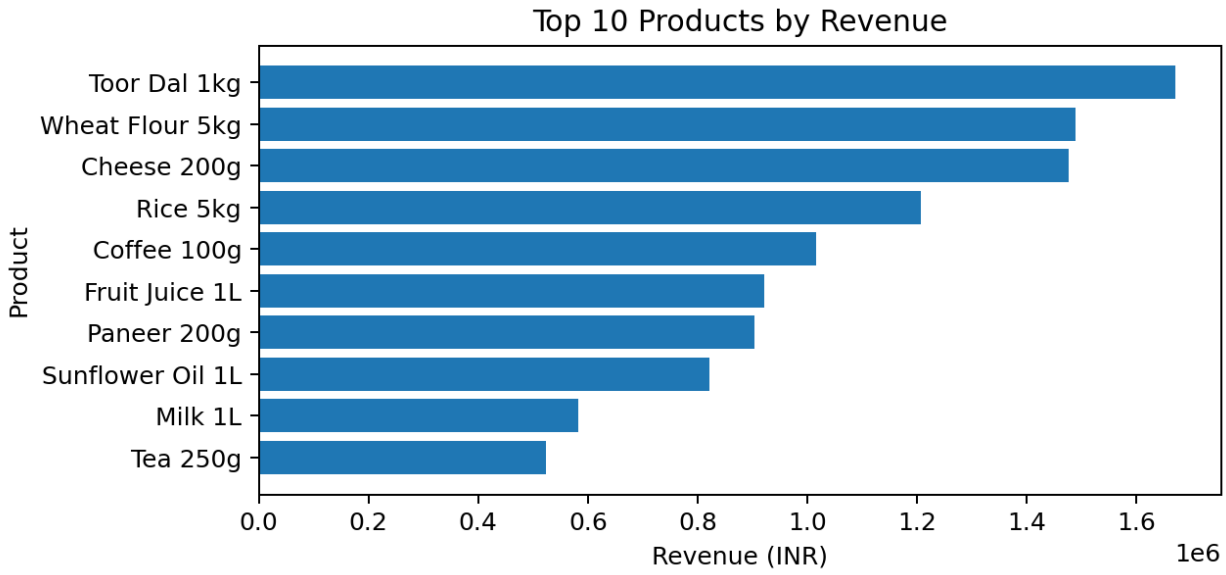


Figure 4. Ten products with the highest recorded revenue.

Product	Category	Revenue (INR)	Units	Unmet
Toor Dal 1kg	Grocery	₹1,671,008	10,582	94
Wheat Flour 5kg	Grocery	₹1,488,110	5,802	67
Cheese 200g	Dairy	₹1,476,163	11,516	194
Rice 5kg	Grocery	₹1,207,836	3,400	46
Coffee 100g	Beverages	₹1,015,350	5,883	92
Fruit Juice 1L	Beverages	₹921,162	8,489	95
Paneer 200g	Dairy	₹904,460	8,728	206
Sunflower Oil 1L	Grocery	₹821,250	5,547	83

Use the ranking to identify candidates for review, then cross-check gross margin estimates, stock records and unmet demand. A high-revenue item may already be well supplied; a lower-revenue item with recurring unmet demand could also matter for customer availability.

Weekday Demand Pattern

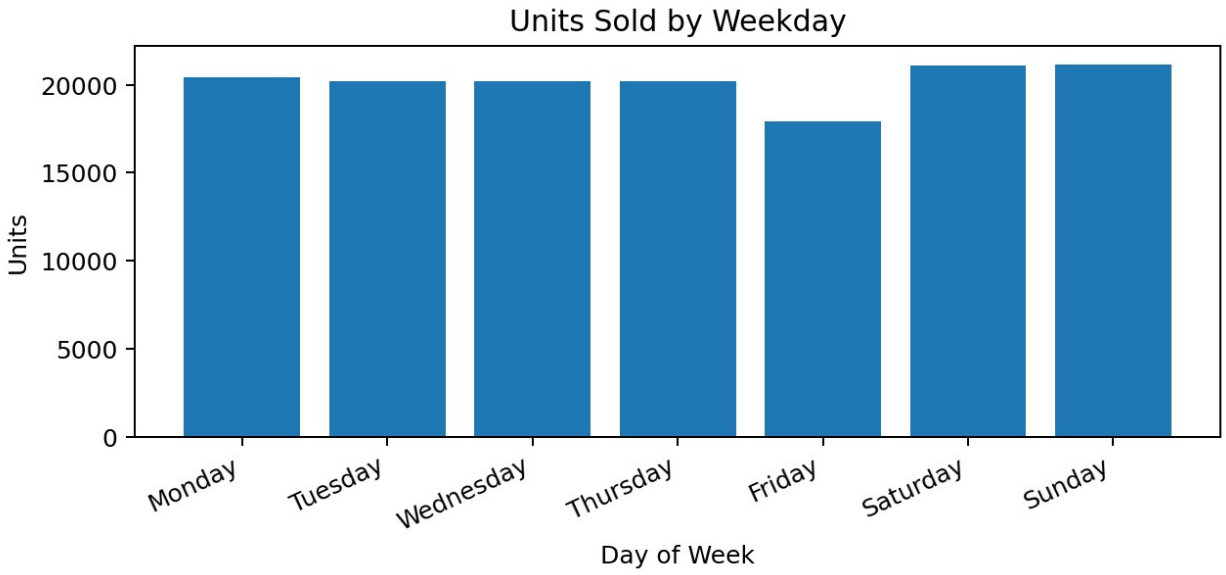


Figure 5. Total units sold by weekday across the 60-day window.

The weekday chart compares total units across weekdays. Because the observation window includes different counts of each weekday and the data is synthetic, compare average units per weekday occurrence as a follow-up analysis rather than relying only on total units.

Recommended follow-up

- Compute average units per date for each weekday.
- Break weekday averages down by category and city.
- Use a validation period to check whether weekday patterns persist.
- Avoid assuming that calendar effects are stable beyond the observed period.

Promotion-Labelled Rows: Descriptive Comparison

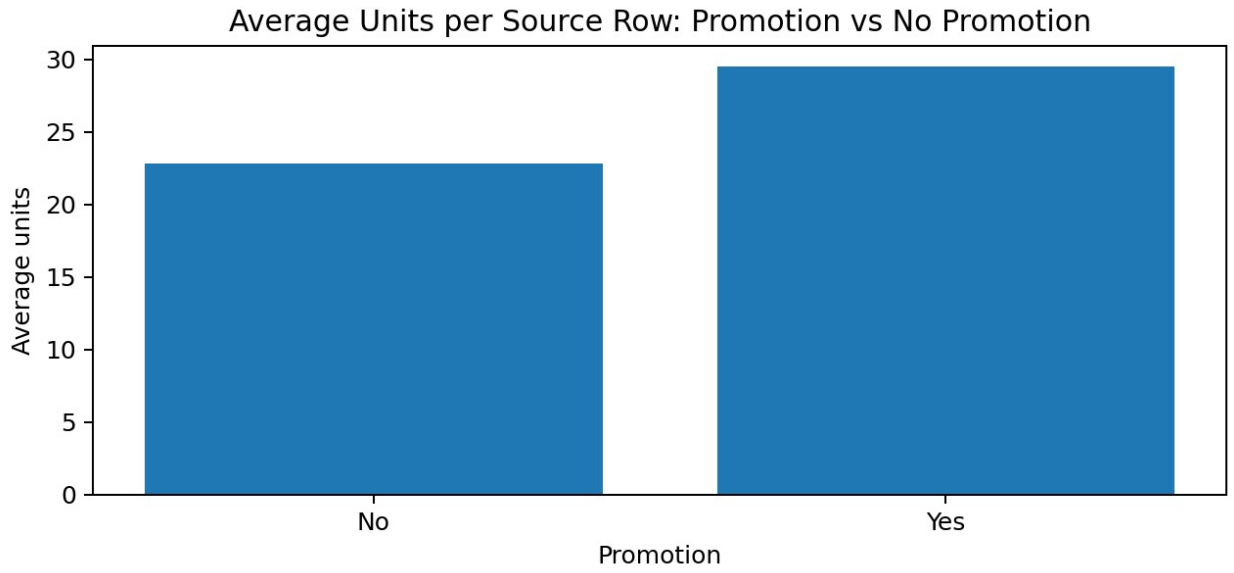


Figure 6. Mean units sold per source row by promotion label.

Promotion label	Source rows	Average units / row	Average revenue / row
No	5,352	22.81	₹2,289.75
Yes	648	29.47	₹2,645.83

The row-level average comparison gives a descriptive difference of 29.2% for promotion-labelled versus non-promotion rows. This is not a causal promotion lift: products, dates, pricing and stock conditions may differ between groups.

For stronger analysis, compare the same product-store combinations across comparable dates, control for weekday and product mix, and use a pre/post or matched design if the dataset supports it.

Stock, Availability & Unmet Demand

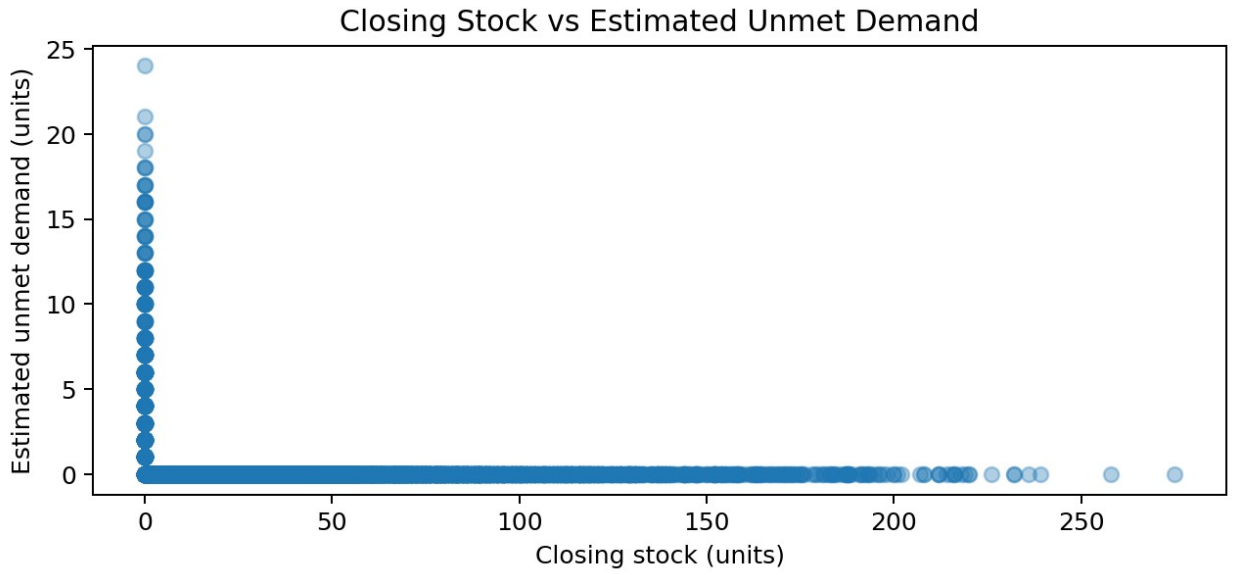


Figure 7. Closing stock plotted against the supplied estimated unmet-demand field.

The workbook reports 2,015 units of estimated unmet demand across 6,000 rows. This is a supplied field and its operational definition is not independently documented. It should be treated as a signal for investigation rather than verified lost sales.

Recommended exception queue

- Filter records with Estimated Unmet Demand > 0.
- Sort by unmet demand, then review closing stock and restocked units.
- Group repeated exceptions by product and store to find persistent patterns.
- Validate stock continuity before estimating a reorder quantity.
- Set a low-stock threshold only after considering lead time and service-level policy.

The scatter plot should not be interpreted as proof that closing stock causes unmet demand. Stock may be measured at a different point in time, and the source's inventory fields may not follow a complete stock-flow identity.

Forecasting Objective & Feature Design

The prediction target is units sold at the daily store-product level. Calendar features, promotion label, store, product and category are used as predictors. The project applies a Random Forest regressor as a baseline because it can model non-linear relationships without requiring a linear functional form.

Feature family	Example	Reason to include	Risk
Calendar	Day number, weekday, month, weekend	Captures repeating calendar structure	Only two months limits seasonal learning
Store	Store ID	Captures store-level differences	Synthetic store effects may not generalize
Product	Product ID, category	Captures product mix	Sparse or changing product patterns
Promotion	Yes/No flag	Potential association with demand	Promotion assignment may be confounded

Evaluation design

The project uses an 80/20 chronological holdout: earlier observations for training and later observations for testing. A time-ordered split is preferable to a random split for forecasting because future dates should not inform the model fitted on past dates.

The feature set is deliberately compact. Before production use, add lagged demand and rolling features computed strictly from past observations, validate them with rolling-origin backtests and compare against simple baselines such as last-week same-day demand.

Model Evaluation & Meaning

Metric	Recorded result	Interpretation
MAE	4.203 units	Average absolute error per evaluated row
RMSE	5.625 units	Penalizes larger errors more heavily
MAPE on non-zero actuals	17.86%	Excludes rows with zero actual units
Training rows	4,800	Earlier observations
Test rows	1,200	Later observations

These values are the metrics recorded in the project's output file. Re-run the pipeline in the submission environment before treating them as final. Compare with a naive baseline; a model score has little value unless it improves on a transparent benchmark.

Limitations

- The two-month synthetic dataset cannot validate annual seasonality.
- MAPE can be unstable for small actual values and excludes zero-actual rows.
- Metrics are aggregate and may hide poor performance for individual products or stores.
- No causal conclusion about promotions or inventory policies follows from forecast accuracy.

Databricks & PySpark Processing Plan

Databricks is the distributed compute environment; PySpark expresses transformations over Spark DataFrames. The notebook should read a known input location, validate schema and keys, transform column names/types, aggregate as required and write outputs to Delta format.

Implementation sequence

16. Upload or mount the source CSV/workbook export using the workspace-approved mechanism.
17. Read source data into a Bronze DataFrame with a captured ingestion timestamp.
18. Apply schema and quality checks; standardize column names and date types in Silver.
19. Create Gold aggregates for city/category/date and store-product demand.
20. Write tables in Delta format and verify counts and schema after the write.
21. Record cluster/runtime and workspace details, but never publish tokens or credentials.

Execution status

The notebook source is included in the project package. Unless a successful run and output evidence are captured in the user's Databricks workspace, describe this section as the planned implementation, not a completed cloud execution.

Delta Lake & Medallion Layers

Layer	Content	Quality rule	Consumer
Bronze	Raw ingested records	Preserve source values; add ingestion metadata	Audit and replay
Silver	Standardized, validated records	Typed dates, consistent names, valid keys and documented rejects	Analytics engineering
Gold	Business aggregates and forecast outputs	Reconciled metrics, documented grain and KPI definitions	Power BI / decision support

Delta Lake adds a transaction log and table-management capabilities to data lake storage. In this design, Bronze preserves the source, Silver standardizes it and Gold serves validated aggregates. The design should be kept simple: three layers are useful only if each layer has a clear contract and validation checks.

Tests for each layer

- Record count and date range should reconcile to the expected input.
- Primary keys or intended grain should be checked for duplicates.
- Reject or quarantine invalid records with an explained reason.
- Repeated notebook runs should not silently double-count the same input.
- Document table paths, access controls and retention decisions.

Power BI Dashboard Specification

Page	Primary question	Suggested visuals	Slicers
Executive Overview	What is happening overall?	Revenue, units, estimated profit and unmet-demand cards; daily trend	Date, city
Demand Analysis	Where is demand concentrated?	Revenue by city/category, top products, weekday trend	Date, category, product
Inventory Exceptions	Where should an analyst investigate?	Unmet demand table, closing-stock chart, store-product matrix	Store, product, city
Promotion Analysis	How do promotion-labelled rows compare?	Average units/revenue by promotion, product-level comparison	Date, promotion, category

Visual design principles

- Use a consistent INR format and clearly labelled units.
- Use cards for headline KPIs and charts for comparison or trends.
- Ensure slicers filter all relevant visuals and date axes sort chronologically.
- Add tooltips and drill-through only where they answer a clear question.
- Display data freshness and note whether refresh is manual or automated.

Do not claim the dashboard is completed until the report has been built in Power BI and screenshots show the actual visuals and filters.

Power BI Dashboard — Executive View

Figure 1 presents a report-ready dashboard prototype generated from the supplied synthetic retail workbook. It consolidates headline KPIs with revenue trends, category performance, leading products, weekday demand and city-level revenue.

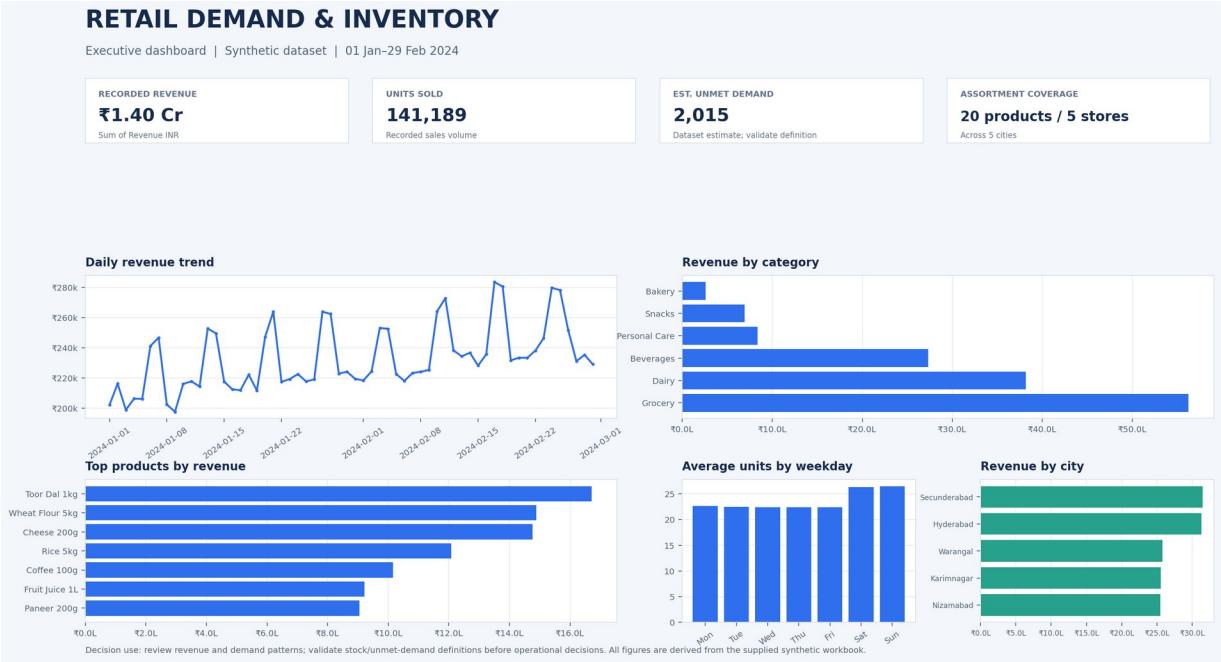


Figure 1. Retail demand and inventory executive dashboard prototype. Revenue and sales volumes describe the supplied synthetic dataset; estimated unmet demand is a modelled/derived indicator and should be validated before operational use.

Suggested Power BI slicers: Date, City, Store ID, Category, Product Name and Promotion. This image is a static prototype for the report; it is not evidence of a published interactive .pbix dashboard.

KPI Dictionary & DAX Guidance

Measure	Definition / formula idea	Display
Total Revenue	SUM of Revenue INR	Currency: INR
Units Sold	SUM of Units Sold	Whole number
Estimated Gross Profit	SUMX(rows, Revenue INR − Units Sold × Unit Cost INR)	Currency; label as estimate
Estimated Unmet Demand	SUM of Estimated Unmet Demand	Whole number; source estimate
Average Selling Price	DIVIDE([Total Revenue],[Units Sold])	INR per unit
Promotion Row Share	Promotion rows / all rows	Percentage; label as row share

Measure quality

The Power BI measures should reconcile to SQL totals after applying the same filters. A measure for gross profit must multiply unit cost by units sold at row level before summing; averaging row-level margins or summing unit costs would be incorrect.

Recommended dashboard checks

- Confirm total revenue and units against SQL and Excel.
- Test each slicer independently and in combination.
- Check blank/zero handling and currency formatting.
- Confirm that totals respond correctly to date and city filters.
- Capture screenshots of each page for the report appendix.

Recommendations & Implementation Priorities

Priority	Recommendation	Why it follows from the data	What to measure
1	Create a daily exception list for non-zero unmet-demand records	The source includes an unmet-demand estimate worth validating	Exceptions reviewed; verified stockout rate
2	Prioritize high-revenue products for deeper store-level review	Revenue is concentrated across products/categories	Availability and gross margin by SKU/store
3	Compare promotion rows within comparable product-store-date groups	Raw promotion averages may be confounded	Adjusted difference; margin after discount
4	Run forecast backtests before replenishment pilots	Current model is only a baseline	MAE/RMSE versus naive baseline
5	Add lead time and order constraints before reorder logic	These operational inputs are absent	Fill rate, holding cost, stockout cost

30-day pilot outline

- Week 1: validate source and inventory definitions with operations.
- Week 2: reconcile SQL/Power BI and publish a daily exception view.
- Week 3: backtest baseline and naive forecast by product/store.
- Week 4: conduct a limited human-reviewed replenishment pilot; compare against a control or prior period.

Do not claim financial benefit in advance. Establish a baseline and define success thresholds before the pilot.

Limitations, Risks & Mitigations

Risk	Potential impact	Mitigation
Synthetic data	Findings may not reflect a real retailer	State provenance; validate methods on real data before decisions
Short time horizon	Weak seasonality inference	Collect at least 12–24 months where available
Ambiguous stock fields	Incorrect stockout interpretation	Confirm timing and stock-flow definitions
Promotion confounding	Overstated promotion impact	Use matched or controlled analysis
Model drift / weak baseline	Forecasts fail in changed conditions	Backtest, monitor and compare with naive forecasts
Cross-tool mismatch	Inconsistent dashboards	Maintain KPI dictionary and reconciliation tests
Credential leakage	Unauthorized access	Use secrets/environment variables; exclude credentials from Git

Responsible use

Forecasts should support a human review process. The project should not trigger automated orders based solely on a short synthetic dataset. Any operational rollout needs ownership, monitoring, rollback procedures, access controls and a clear way to correct bad data.

Conclusion & Future Work

This project provides a structured path from a spreadsheet to a reproducible retail analytics workflow. The supplied workbook supports descriptive analysis of revenue, unit demand, product/category mix, promotion labels and estimated unmet demand. A compact Python baseline creates a starting point for forecast evaluation, while PostgreSQL and Power BI provide a relational and decision-facing layer.

The central finding is methodological: useful forecasting depends on trustworthy data definitions, a correct observation grain, chronological evaluation and KPI reconciliation—not merely on choosing a machine-learning algorithm or a dashboard tool.

Future work

- Replace synthetic data with approved, operational retail history.
- Validate inventory continuity and document the meaning of unmet demand.
- Add lag/rolling demand features and rolling-origin evaluation.
- Compare with seasonal-naive and moving-average baselines.
- Incorporate supplier lead time, minimum order quantity and service-level targets.
- Run the Databricks notebook and capture actual Delta table evidence.
- Finish Power BI pages and reconcile all KPIs to PostgreSQL.
- Publish a clean GitHub repository with setup instructions and no secrets.

Final assessment

The prototype is suitable for demonstrating an integrated analytics approach when the execution status of each component is stated accurately. It is not yet evidence of production readiness or verified financial improvement.

Reproducibility & Submission Checklist

Item	Evidence to include	Status before submission
Dataset profile	Source sheets, rows, columns, date range, nulls and duplicate checks	Verify in own environment
PostgreSQL / DBeaver	Schema, imported tables, saved queries and reconciled totals	Capture real screenshots
Python	Run command, dependencies, metrics and prediction CSV	Re-run and save outputs
Databricks / Delta	Notebook run, table schema/path and row counts	Only mark complete after successful run
Power BI	Four pages, KPI cards, 4+ visuals, slicers and refresh status	Capture actual dashboard screenshots
GitHub	README, SQL, Python, notebook, DAX, report and approved data	Verify repository URL/access
Report	Student/faculty details, page count, figure captions and appendix	Check final Word/PDF pagination

Reproducibility commands

```
pip install -r requirements.txt
python python/pipeline.py --input data/retail_sales_data.csv --output outputs
```

Final note: use only actual screenshots and actual execution outputs. Replace all bracketed cover-page details. Keep the original synthetic-data statement and report any unresolved quality issue.